

APARA Simulator (engine_isp) Bug Report Solved and Unsolved Defects, With Source

APARA C Compiler Project — Verification & Audit

June 21, 2026

Contents

1	Introduction and Methodology	2
1.1	Scope	2
1.2	Authoritative source and binary	2
1.3	How these were found	2
2	Solved Bugs	4
2.1	Bug E1: \$ld sub-word loads ignored the real byte offset, always reading the upper half-word	4
2.2	Bug E2: \$call's disassembler sign-extended the wrong bit, breaking every backward function call	5
2.3	Bug E3: instruction memory was half the documented size, silently truncating larger programs	6
3	Unsolved Bugs	8
3.1	Bug E4 (major, correctness): \$vreduce on unsigned vector types sign-extends instead of zero-extending	8
3.2	Bug E5 (major, correctness): 4-bit vector types (vi4/vu4) silently compute garbage	10
3.3	Bug E6 (minor, diagnostic-only): misleading error messages in the post-condition file parser	11
4	Related Findings (Not Source-Code Bugs)	13
4.1	Two disagreeing simulator binaries exist in the project tree	13
4.2	Two diverged simulator source checkouts caused transient confusion	13
4.3	Function pointers are blocked by a hard architectural limitation, not a bug . . .	13
5	Summary	14

Chapter 1

Introduction and Methodology

This report documents every confirmed defect found in the APARA hardware simulator’s own source code (`engine_isp`’s C++ assembler/simulator, *not* the Python C compiler being developed alongside it) across the full history of this project. Each entry was confirmed by checking that the compiler emitted the textually-correct mcode instruction for the case in question, and that the simulator’s own assembly or execution of that *correct* instruction is what produced the wrong result — ruling out a compiler bug before ever blaming the simulator.

1.1 Scope

Six confirmed defects are documented: three have been found, root-caused, fixed, and verified as currently present in the authoritative simulator binary; three remain confirmed but unfixed, and are flagged here for the course staff/hardware team, since they sit outside what a C-compiler project can fix on its own. Three further findings are *not* source-code bugs but are directly relevant environment/tooling issues encountered while building this verification: two disagreeing simulator binaries in the project tree, a divergence between two simulator source checkouts, and a hard architectural limitation (not a bug) blocking function pointers.

1.2 Authoritative source and binary

All line numbers and code listings in this report are read directly from the source tree backing the project’s authoritative simulator binary, `mcode_run`, confirmed by matching file timestamps and verified against the project’s own build records. All “current status” claims (fixed/not fixed, present/absent) were re-checked against this live source immediately before writing this report — not assumed from memory.

1.3 How these were found

The three solved bugs (Chapter 2) surfaced from a standalone investigation into nested function calls returning garbage: any C function called from within another function, where the callee was defined *before* its caller in source order (the ordinary C pattern), returned a stale or zero value instead of computing correctly. Six separate compiler-side hypotheses — bundle padding, jump-target resolution, bundle shape/ordering, caller-save/restore ordering, callee-internal codegen, and epilogue scratch-register clobber — were checked and individually ruled out before the investigation moved into the simulator’s own source and found the real causes there.

The three unsolved bugs (Chapter 3) surfaced from a systematic instruction-coverage sweep with an independent “no bias” verification methodology: every test’s expected result is computed by compiling the *same* source natively with `gcc` against a plain-C reference implemen-

tation, never by this project's own compiler or by hand-derivation, and checked against the simulator's real execution via its own built-in `PostCondition` mechanism.

Chapter 2

Solved Bugs

These three bugs were root-caused, fixed, and the fixes are confirmed present in the *current* authoritative simulator source and binary — not just a one-off verification build. All three fixes are exercised by tests that are part of the project’s current, continuously-passing regression baseline.

2.1 Bug E1: `$ld` sub-word loads ignored the real byte offset, always reading the upper half-word

Status: **SOLVED** (fixed upstream by the hardware/VM team; fixed binaries pulled into this project’s simulator build; hardware-confirmed).

Symptom

`$ld ($i32)` always returned bits [63:32] (the upper 32 bits) of the 8-byte DMEM word being addressed, regardless of the byte offset actually encoded in the load instruction. `$st ($i32)` was correct — it wrote to the upper or lower 32 bits depending on the real offset — only the *load* side ignored the offset. Any value stored at byte-offset 4 of a word (the conventional placement for a value that isn’t the first element in that word) could never be read back correctly by a 32-bit load.

Why this mattered

This is the load instruction underlying essentially every `int`, `short`, and `char` scalar or array access compiled from C — i.e. almost everything a real program does.

Workaround used while the bug was live (compiler-side, no longer needed)

Every C element, regardless of type, was given its own full 8-byte-aligned DMEM slot (`stride = max(elem.bytes, 8)`), and the compiler always emitted (`$i64`) loads/stores regardless of the real element width — sidestepping the broken sub-word addressing path entirely rather than relying on it. The compiler’s narrower load/store support was only re-enabled *after* this fix was confirmed on hardware.

Fix and verification

Fixed upstream in the VM/hardware build; the corrected binaries were pulled into this project’s `engine.isp` checkout. A full 19-program hardware regression (`test_subword_i8.c`, `test_subword_i16.c`, isolated single-width probes) both returned `r1=1` (full pass). `test_array`, `test_cmov`, `test_branch`,

and `test_pointer` — all exercising `$i32` in other shapes — were independently confirmed correct on the same hardware run. This fix underlies the compiler’s current `i8/i16/i32` sub-word load/store support and is part of the project’s continuously-passing baseline.

2.2 Bug E2: `$call`’s disassembler sign-extended the wrong bit, breaking every backward function call

Status: SOLVED — confirmed present in the current source (verified by direct inspection immediately before writing this report).

The code, as currently fixed (`McodeDisassemble.cpp`, function `DisassembleToCallInstr`)

Listing 2.1: `McodeDisassemble.cpp`, lines 261–284 (current, fixed state)

```

1 McodeInstruction* DisassembleToCallInstr (uint32_t pc, uint32_t hex_instr)
2 {
3     uint32_t opcode = Get_Slice(31, 26, hex_instr);
4     uint32_t imm_flag = Get_Slice (25,25, hex_instr);
5     uint32_t rs2 = Get_Slice (4,0, hex_instr);
6     int32_t relative_jump = (int32_t) Sign_Extend(24, Get_Slice (24, 0, hex_instr));
7     // FIX: was Sign_Extend(25, ...) -- see explanation below
8
9     McodeCallInstruction* ci = NULL;
10    if(imm_flag)
11    {
12        int32_t label_address = ((int32_t) pc) + relative_jump;
13        assert(label_address >= 0);
14
15        string lbl = "l_" + Int64ToString(label_address);
16        ci = new McodeCallInstruction (lbl);
17        ci->Set_Pc_Relative_Jump_Address (relative_jump);
18    }
19    else
20    {
21        ci = new McodeCallInstruction (rs2);
22    }
23    return(ci);
24 }
```

The bug (before this fix)

The line read `Sign_Extend(25, Get_Slice(24, 0, hex_instr))`. `Sign_Extend`’s first argument is a *bit index*, defined as (`McodeUtils.cpp`, line 371):

Listing 2.2: `McodeUtils.cpp`, lines 371–382 — `Sign_Extend`’s definition

```

1 uint32_t Sign_Extend (uint32_t sign_index, uint32_t x)
2 {
3     bool pad_ones = (((1 << sign_index) & x) != 0);
4     uint32_t or_mask = ((~0) << (sign_index+1));
5
6     if(sign_index >= 31)
7         or_mask = 0;
8
9     uint32_t ret_val = pad_ones ? (x | or_mask) : x;
10    return(ret_val);
```

11 }
}

\$call's jump field is 25 bits wide — bits 24:0 — so its real sign bit sits at index **24**, not 25. `Get_Slice(24, 0, hex_instr)` already masks the value down to bits 24:0 *before* `Sign_Extend` is called, so bit 25 of that already-masked value is unconditionally 0 — `pad_ones` can never be true, and negative (backward) call offsets were *never* sign-extended.

Why this mattered

Any function called from inside another function (not `main` itself), where the callee is defined *before* its caller in source order — the completely ordinary C pattern, e.g. a helper `f()` written above `main()` — computes a *backward* jump. With the sign-extension bug, every such backward call computed `target + 225` instead of `target`, landing in zero-filled garbage memory. `f`'s body never executed at all; the caller's return register simply held whatever value happened to already be sitting in it before the call — a silent, plausible-looking wrong answer, not a crash.

Confirmed isolated to \$call, not systemic: `DisassembleToBranchInstr` (same file, line 302, listed above) does `Sign_Extend(11, Get_Slice(16,5,...))` on its own 12-bit field (bits 16:5, real sign bit at index 11) — called with 11, correctly indexed. This is consistent with `while/for` loops, which use `$branch` rather than `$call` for their own backward jumps, having always worked correctly throughout this project.

Verification

- Minimal repro: `int f(void) { return 6; } int main() { return f(); }`. Before the fix, `$call f` resolved to `npc=0x2000018` and never entered `f`; after the fix, disassembly correctly shows `$call 1_24`, `f`'s body executes, and the program halts with `r1=6` exactly as expected.
- `test_struct.c` (which calls helper functions defined above `main`) went from wrong (`0xa`) to exactly correct (`0x0`) with this fix alone.
- Full regression: zero regressions among the other already-passing tests.
- **Currently in production:** `test_struct` is part of the project's current continuously-passing baseline against the authoritative simulator binary, confirming this fix is live, not just a one-off verification build.

2.3 Bug E3: instruction memory was half the documented size, silently truncating larger programs

Status: SOLVED — confirmed present in the current source.

The code, as currently fixed (`McodeClasses.hpp`)

Listing 2.3: `McodeClasses.hpp`, lines 139, 143 (current, fixed state)

```
1 uint32_t __instruction_memory[4096]; // 16KB (AparaReference.pdf p.6, Fig 1.1:
2                                     // "16KB of instruction space", 4 bytes/
3                                     // -> 4096 words). Was [2048] (8KB) -- half the
4                                     // documented size.
5 ...
6 int Instr_Mem_Size_In_Words () {return (4*1024);} // 16 KB, see note above
```

The bug (before this fix)

AparaReference.pdf, p. 6, §1, Figure 1.1, states explicitly: “*The instruction memory provides 16KB of instruction space to each accelerator. Each instruction is 4-bytes.*” $16\text{KB} \div 4 \text{ bytes} = 4096 \text{ words}$. The simulator’s array was sized [2048] — half the documented capacity, carrying a comment that claimed “16KB” but provided only 8KB.

The instruction loader (McodeAccelerator.cpp’s Init_Instruction_Memory) silently **drops** — logs an **Error:**, does not write, does not abort — any instruction whose program counter is ≥ 2048 . A program larger than 2048 words runs straight off the end of its own truncated body into zero-filled memory, with no crash and no obvious symptom: execution simply continues into \$null until the simulator’s tick budget runs out, leaving the return register frozen on whatever value it last held, which can coincidentally look like a plausible (but wrong) answer.

Why this mattered

Two real test programs were large enough to trigger it: `test_scalar_full` (2688 words; 640 silently dropped) and `test_spill` (2496 words; 448 dropped). Both were initially misdiagnosed as having a remaining logic bug, when in fact their actual return-value computation was simply never loaded into memory at all.

An intermediate false lead, corrected before finalizing

A first attempt bumped the array further than strictly necessary (to [16384]) as a verification-only change, and was briefly reverted on the strength of a verbal claim that “2048 words is the real hardware limit.” That claim was then checked directly against the written ISA specification (cited above) and found to be incorrect — 4096 words, not 2048, is the documented figure. The fix shown above uses the spec-correct value, not the over-sized verification value.

Verification

Table 2.1: Before/after, exact register values

Test	Before fix	After fix	Expected
<code>test_scalar_full</code>	0x7ff0 (stale leftover)	0xc	0xc (12)
<code>test_spill</code>	0x19 (stale leftover)	0x1d1	0x1d1 (465)

Combined with Bug E2’s fix, all three originally-broken tests (`test_struct`, `test_spill`, `test_scalar_full`) now produce exactly their expected values. Full regression: zero regressions among the other 14 tests. **Currently in production:** both tests are part of the project’s current continuously-passing baseline.

Chapter 3

Unsolved Bugs

These three bugs are confirmed, precisely root-caused, and reproduced — but not fixed. They are flagged here for the course staff/hardware team, since fixing C++ simulator source is outside the scope of the Python C-compiler project these were found while building.

3.1 Bug E4 (major, correctness): \$vreduce on unsigned vector types sign-extends instead of zero-extending

Status: NOT SOLVED.

The code (exact, current, unfixed)

Listing 3.1: McodeOperations.cpp, lines 110, 146–198 — `__vreduce_operation__`

```
1 void __vreduce_operation__ (McodeOpcode sub_opcode, McodeType dest_type,
2                             McodeType src_type, vector<uint64_t>& rs1_in_vector,
3                             uint64_t& ovalue)
4 {
5     // ... (lines 112-145: setup + the float-type branch, not relevant here)
6
7     for(I=0, fI = rs1_in_vector.size(); I < fI; I++)
8     {
9         uint64_t ele = rs1_in_vector[I];
10
11         // sign extend to 64-bits.
12         int64_t r = (int64_t) Sign_Extend_64(src_type.Get_Nbits()-1, ele); // <-- BUG
13         : see below
14         if(signed_flag)
15         {
16             int64_t r = (int64_t) Sign_Extend_64(src_type.Get_Nbits()-1, ele);
17             if(sub_opcode == __ADD)
18                 s_result = s_result + r;
19             // ... (same pattern for MUL/OR/AND/XOR/XNOR/MAX/MIN)
20         }
21         else
22         {
23             if(sub_opcode == __ADD)
24                 result = result + r; // <-- uses the OUTER, sign-extended r
25             // ... (same pattern for MUL/OR/AND/XOR/XNOR/MAX/MIN)
26         }
27     }
28     ovalue = CastToU64 (signed_flag, dest_type.Get_Nbits(),
```

```

28         (signed_flag ? ((uint64_t) s_result) : result));
29     }

```

What's wrong

The line marked first `// <-- BUG` unconditionally sign-extends `ele` into `r`, *before* the `if(signed_flag)` branch is even reached — so it runs for every element regardless of the vector's actual declared signedness.

- The `if(signed_flag)` branch redeclares its own local `r`, identical to the outer one — correct, just redundant.
- The `else (unsigned)` branch declares **no local `r` at all**. Every use of `r` inside it falls through to the outer, already-sign-extended `r`. The unsigned path never zero-extends `ele`; it silently reuses the signed value computed for the wrong case.

Why this matters

For an unsigned element with its top bit set (e.g. a `$vu8` byte `0xFC = 252`), the correct behavior is to zero-extend it to 252 before summing. The bug sign-extends it to `-4` instead — the simulator computes the *signed* interpretation for a value the program declared *unsigned*.

Confirmed reproduction

Listing 3.2: Reproduction, exact numbers

```

1 vector (vu8, 8 elements): [1, 2, 3, 0xFC, 5, 6, 7, 8]
2   $vreduce + on ($vi8)  -> 28      (correct, signed: 1+2+3+(-4)+5+6+7+8)
3   $vreduce + on ($vu8)  -> 28      (simulator's actual output -- WRONG, should be 284)
4                                   (correct, unsigned: 1+2+3+252+5+6+7+8 = 284)

```

Same pattern confirmed at `$vu16` (expected 65538, got 2) and `$vu32` (expected 4294967295, got -1) — generic across all three unsigned vector widths, not width-specific. Currently causes exactly 3 documented/expected errors in this project's verification baseline, landing precisely on these three cases. A standalone reproduction program independently confirms the same 3-error pattern against the real simulator binary.

Suggested fix

Listing 3.3: Minimal fix

```

1 else
2 {
3     uint64_t r = ele & __mmask__(src_type.Get_Nbits()); // zero-extend, not sign-
4                                                         extend
5     if(sub_opcode == __ADD)
6         result = result + r;
7     // ... (same for the remaining sub-opcodes)
8 }

```

`__mmask__` already exists and is used elsewhere in this same file, so no new helper is needed.

3.2 Bug E5 (major, correctness): 4-bit vector types (vi4/vu4) silently compute garbage

Status: NOT SOLVED — explicitly deprioritized, not used frequently in this project's target workloads.

The code (exact, current, unfixed)

Listing 3.4: McodeOperations.cpp, lines 50–74 — CastToU64

```

1 uint64_t CastToU64 (int signed_flag, uint32_t nbits, uint64_t ival)
2 {
3     uint64_t result;                // <-- never initialized
4     switch(nbits)
5     {
6         case 8:
7             if(signed_flag) result = ___signed_cast___(int8_t, ival);
8             else result = ___unsigned_cast___ (uint8_t, ival);
9             break;
10        case 16:
11            if(signed_flag) result = ___signed_cast___(int16_t, ival);
12            else result = ___unsigned_cast___ (uint16_t, ival);
13            break;
14        case 32:
15            if(signed_flag) result = ___signed_cast___(int32_t, ival);
16            else result = ___unsigned_cast___ (uint32_t, ival);
17            break;
18        case 64:
19            if(signed_flag) result = ___signed_cast___(int64_t, ival);
20            else result = ___unsigned_cast___ (uint64_t, ival);
21            break;
22        // <-- no "case 4:", and no "default:" either
23    }
24    return(result);
25 }
```

What's wrong

The `switch(nbits)` has cases for **8, 16, 32, 64 only**. For any 4-bit-wide result — every `vi4/vu4/vf4` vector element operation — the switch falls through with no case matching. **result** is declared but **never assigned**, so the function returns whatever uninitialized garbage value happened to be sitting on the stack at that point.

Confirmed reproduction (hand-computed, reproducible across repeated runs)

Table 3.1: 4-bit vector add: expected vs. actual hardware result

Type	Expected (hand-computed)	Hardware actual	Result
vu8 (control)	0x1112131415161718	0x1112131415161718	exact match
vi4	0x3333333333333333	0x0	WRONG

(vi4 case: a=0x1111111111111111, b=0x2222222222222222, every nibble 1+2=3, no overflow possible — yet the hardware returns exactly 0.)

Type parsing itself is correct (the grammar correctly resolves `vi4_t` to `nbits=4`); the bug is purely the missing switch case, not type recognition.

Scope check — audited, not guessed

Every *other* `switch(nbits)` block elsewhere in the simulator (in `McodeNumeric.cpp` and five blocks in `McodeFpuUtils.cpp`) either explicitly handles 4 bits, or fails loudly with `assert(0)` instead of silently returning garbage. `CastToU64` is the *only* one that declares its result variable uninitialized with no `default`: case at all — confirmed isolated to this one function, not a systemic pattern across the codebase.

Suggested fix

Add a `case 4`: doing a manual 4-bit sign-extend/mask (there is no native C++ `int4_t` to reuse the existing `___signed_cast___`/`___unsigned_cast___` macros with directly).

Status

`vu4/vf4` are untested but share the identical code path, so should be assumed equally broken until checked. This project's own instruction-coverage scope explicitly excludes `vi4/vu4` for exactly this reason.

3.3 Bug E6 (minor, diagnostic-only): misleading error messages in the post-condition file parser

Status: NOT SOLVED — does not affect execution correctness, only diagnostic clarity.

The code (exact, current, unfixed)

Listing 3.5: `McodeAccelerator.cpp`, `Verify_Line`, relevant excerpts (lines 422–516)

```

1  else if (tokens[1] == "reg")
2  {
3      //      reg <thread-id>  reg-id reg-value [reg-mask]
4      if (tokens.size() < 3)
5      {
6          McodeRoot::Error ("Incomplete reg line in Verify_Line", this);    // line 427,
          correct here
7          ret_val = 1;
8      }
9      ...
10 }
11 else if (tokens[1] == "mem")
12 {
13     //      mem <address>      mem-value [mem-value-mask]
14     if (tokens.size() < 4)
15     {
16         McodeRoot::Error ("Incomplete reg line in Verify_Line", this);    // line 473
          -- says "reg", wrong branch
17         ret_val = 1;
18     }
19     ...
20 }
21 else
22 {

```

```
23     ret_val = 1;
24     McodeRoot::Warning("Unknown verify line keyword " + tokens[0], NULL); // line 515
    -- checks tokens[1], reports tokens[0]
25 }
```

Bug E6a — wrong keyword named in the “incomplete mem line” error (line 473)

This branch is reached only when parsing a malformed **mem** line, but the message says “**Incomplete reg line**” — copy-pasted from the **reg**-handling branch (line 427) a few lines above and never updated for its new home.

Bug E6b — error message names the wrong token on an unrecognized line (line 515)

Every branch in this function dispatches on `tokens[1]` (the file format is `<thread-id> <keyword> <args...>`, e.g. `0 mem 0x80 0x1`). But the fallback warning at line 515 prints `tokens[0]` (the thread-id) instead of the keyword that actually failed to match. Feeding a line in a slightly different but plausible-looking format (e.g. missing the leading thread-id, so what should be the keyword shifts into `tokens[0]` and the real `tokens[1]` becomes something else) lands in this branch and reports something like `Unknown verify line keyword reg` — which reads as though “**reg**” itself were an invalid keyword, when “**reg**” is valid; the real problem is its position in the line.

Suggested fix

Listing 3.6: Two one-line fixes

```
1 McodeRoot::Error ("Incomplete mem line in Verify_Line", this); // for E6a
2 McodeRoot::Warning("Unknown verify line keyword " + tokens[1], NULL); // for E6b
```

Status

Not fixed. Diagnostic-only — does not affect any verification check’s actual pass/fail correctness, only the clarity of error messages encountered while reverse-engineering the exact file format this parser expects.

Chapter 4

Related Findings (Not Source-Code Bugs)

4.1 Two disagreeing simulator binaries exist in the project tree

A second, different `mcode_run` binary exists elsewhere in the project tree, confirmed (by MD5 checksum) to be a different build from the authoritative one used throughout this project. It accepts a *different* `PostCondition` file format (no leading thread-id) and belongs to a separate RTL/testbench-oriented verification setup — not this project’s compiler-targeted simulator. Despite having a *newer* file timestamp than the authoritative binary, file recency alone is not evidence of which build is correct for this project; several pre-existing example result files elsewhere in the tree appear to have been written against the non-authoritative binary’s format and will not verify correctly against the authoritative one.

4.2 Two diverged simulator source checkouts caused transient confusion

While testing Bugs E2/E3’s fixes, a local rebuild was discovered to diverge from the historical baseline checkout in roughly 15 source files unrelated to either fix. This caused two previously-passing tests to newly fail when run against the diverged toolchain, even though they passed against the original checkout on the exact same compiled object file — not a regression from either fix, just evidence that the two trees are genuinely different codebases, not “the same engine plus two patches.” Resolved by consolidating this project on a single authoritative binary; both affected tests are part of the current passing baseline against that binary.

4.3 Function pointers are blocked by a hard architectural limitation, not a bug

There is no instruction that can load a function’s absolute address into a register. `$call <label>` encodes a PC-relative offset, not an absolute address. `$set`’s immediate-field grammar (confirmed directly against the assembler’s own parser source) only accepts numeric literal tokens — a label name in that position is a hard parse failure, and the assembler has no label-resolution mechanism outside of control-transfer instructions. This is categorically different from the bugs in this report: it is not broken logic to fix, it is a missing capability that would need a new instruction or assembler feature. The Python compiler’s own IR/codegen support for function pointers is fully built and ready; it is blocked purely by this assembler-level gap, and cannot be worked around from the compiler side.

Chapter 5

Summary

Table 5.1: All six confirmed `engine_isp` bugs, at a glance

#	Bug	File	Status
E1	<code>\$ld</code> sub-word loads ignored the real byte offset	DMEM load logic (upstream VM)	SOLVED
E2	<code>\$call</code> disassembler sign-extended the wrong bit	<code>McodeDisassemble.cpp:266</code>	SOLVED
E3	Instruction memory half the documented size	<code>McodeClasses.hpp:139,143</code>	SOLVED
E4	<code>\$vreduce</code> unsigned sign-extends instead of zero-extends	<code>McodeOperations.cpp</code> (<code>__vreduce_operation__</code>)	NOT SOLVED
E5	<code>vi4/vu4</code> vector ops return garbage	<code>McodeOperations.cpp</code> (<code>CastToU64</code>)	NOT SOLVED
E6	Misleading <code>VerifyLine</code> error messages	<code>McodeAccelerator.cpp:473,485</code>	NOT SOLVED (diagnostic-only)

Of six confirmed defects in the simulator’s own source, three are fixed and verified present in the currently-used authoritative binary (E1–E3), and three remain open, confirmed, precisely root-caused, and reproduced, with suggested one-paragraph fixes each (E4–E6). None of the three open bugs are fixable from the C-compiler side of this project — E4 and E5 require a C++ source change in the simulator itself, and E6 is a pure diagnostics fix in the same codebase.